

SENA · ADSO 2026

Diseño Técnico del Proyecto

Diseño Técnico del Proyecto

Texticode — Sistema de Gestión Textil

API de Google — Login y Calendar (App Móvil)

Aprendices

Kevin Stevan Wagner

Kevin Andrés Castro

Andrés Felipe Lancheros

Camilo Andrés Tibambre

Instructor

Ing. Edwin Marín

Fecha: Septiembre 2026

Programa: ADSO 2026

1. Arquitectura 3 Capas

Texticode Mobile implementa una arquitectura REST desacoplada en 3 capas. La app Flutter nunca habla directo con Google Calendar ni maneja el `client_secret` o el `refresh_token`: solo obtiene el `idToken` y el `serverAuthCode` con el SDK nativo de Google Sign-In y los envía al backend, que se comunica con Google y responde con el JWT de Texticode.

Capa	Detalle
FRONTEND	Flutter · google_sign_in · GoogleAuthRepository · CalendarRepository · perfil (admin, operario, cliente) · GOOGLE_WEB_CLIENT_ID por <code>--dart-define</code>
BACKEND	Node.js + Express (mismo backend de la web) · <code>/api/auth/google/mobile</code> · <code>/api/calendar/*</code> · JWT (<code>JWT_SECRET</code> , <code>JWT_EXPIRES=8h</code>) · <code>GOOGLE_CLIENT_ID</code> y <code>GOOGLE_CLIENT_SECRET</code> en <code>.env</code>
GOOGLE	Google Sign-In (<code>idToken</code>) · OAuth 2.0 (intercambio del <code>serverAuthCode</code> sin <code>redirect_uri</code>) · Google Calendar API v3 con scope <code>calendar.events</code> · el <code>refresh_token</code> queda guardado en el servidor asociado al <code>Id_Usuario</code>

Flujo de datos — iniciar sesión con Google y vincular Calendar

Capa	Acción
Frontend	<code>GoogleSignIn.signIn()</code> (scopes <code>email + calendar.events</code>) → obtiene <code>idToken</code> y <code>serverAuthCode</code> → <code>POST /api/auth/google/mobile (2.1)</code>
Backend · Auth	Verifica el <code>idToken</code> contra Google con el Web Client ID, busca al usuario por Correo y valida que su Estado sea activo
Google · OAuth	El backend intercambia el <code>serverAuthCode</code> (sin <code>redirect_uri</code>) y recibe <code>access_token + refresh_token</code>
Backend · Persistence	Guarda el <code>refresh_token</code> asociado al <code>Id_Usuario</code> y firma el JWT de Texticode
Response	200 OK + { token, usuario: { Id_Usuario, Nombre_Completo, Correo, Id_Rol, Rol, Estado } }

Flujo de datos — sincronizar órdenes con Google Calendar

Capa	Acción
Frontend	Botón "Sincronizar Ahora" → <code>POST /api/calendar/sync (2.5)</code> con <code>Authorization: Bearer JWT</code>
Backend · Auth	Valida el JWT y comprueba que el usuario tenga Calendar vinculado y la sincronización activa
Backend · Business	Filtra las órdenes según el rol: admin todas · operario y cliente solo las suyas
Google · Calendar API	Crea o actualiza un evento por cada orden con <code>Fecha_Limite</code> , usando el <code>refresh_token</code> guardado
Response	200 OK + { creados, actualizados, total }

Requisitos previos — Google Cloud Console

Paso	Acción
1	Crear o seleccionar el proyecto de Google Cloud que usa la versión web.

Paso	Acción
2	Habilitar "Google Calendar API".
3	Pantalla de consentimiento OAuth: agregar los scopes openid, email, profile y https://www.googleapis.com/auth/calendar.events , y los correos de prueba (test users).
4	Crear un ID de cliente OAuth tipo "Aplicación web" (es el GOOGLE_WEB_CLIENT_ID; el backend usa el mismo).
5	Crear un ID de cliente OAuth tipo Android con package com.texticode_app.texticode_app y las huellas SHA-1 / SHA-256 del keystore (gradlew signingReport).
6	Ejecutar la app con: flutter run --dart-define=GOOGLE_WEB_CLIENT_ID=<web client id>

Variables del entorno Thunder Client

Variable	Valor / uso
Host	192.168.0.6:3001 (IP del backend; debe coincidir con ApiConstants.baseUrl y estar en la misma red Wi-Fi)
ID_TOKEN	idToken de Google del usuario (aud = Web Client ID). Se obtiene desde la app o con OAuth 2.0 Playground usando el Web Client ID propio; se usa en 2.1
SERVER_AUTH_CODE	Código de un solo uso que entrega el SDK nativo (account.serverAuthCode); se usa en 2.1 y 2.3
GOOGLE_ACCESS_TOKEN	accessToken de Google (opcional; el backend no lo usa para verificar identidad)
JWT	Token de Texticode devuelto en 2.1 (o en /api/auth/login). Va en Authorization: Bearer
ID_ORDEN	Id_Orden de una orden existente con Fecha_Limite; se usa en 2.6

Orden recomendado de ejecución

Paso	Endpoints
1 · Iniciar sesión	2.1 (guardar el token en JWT)
2 · Estado inicial	2.2 (connected: false o true, según si el login ya vinculó Calendar)
3 · Vincular	2.3 con un serverAuthCode nuevo → 2.2 (connected: true)
4 · Sincronizar	2.5 · 2.6
5 · Desvincular	2.4 → 2.2 (connected: false)

2.1 POST /api/auth/login

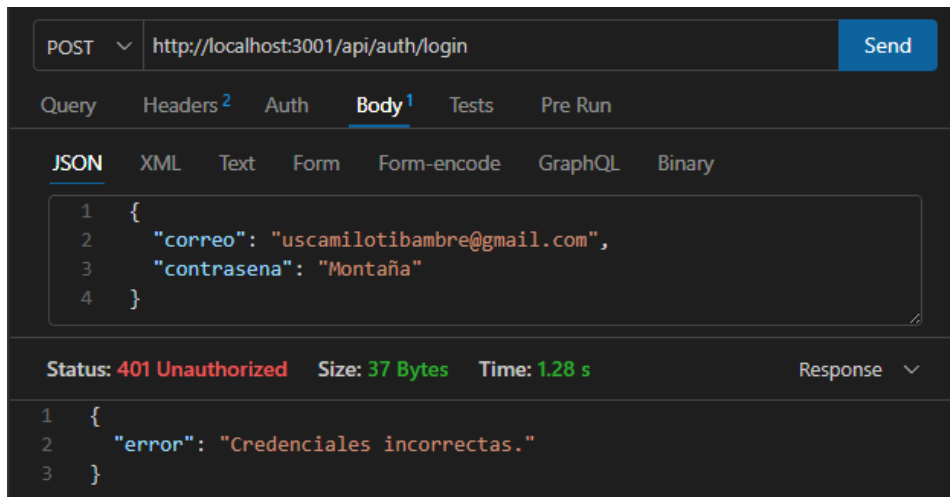
```
/api/auth/login
```

REQUEST

URL completa

EVIDENCIA THUNDER CLIENT





2.2 GET /api/calendar/status

GET

/api/calendar/status

Indica si el usuario autenticado ya vinculó su Google Calendar y con qué correo.

REQUEST

```

GET /api/calendar/status HTTP/1.1
Host: 192.168.0.6:3001
Authorization: Bearer {{JWT}}

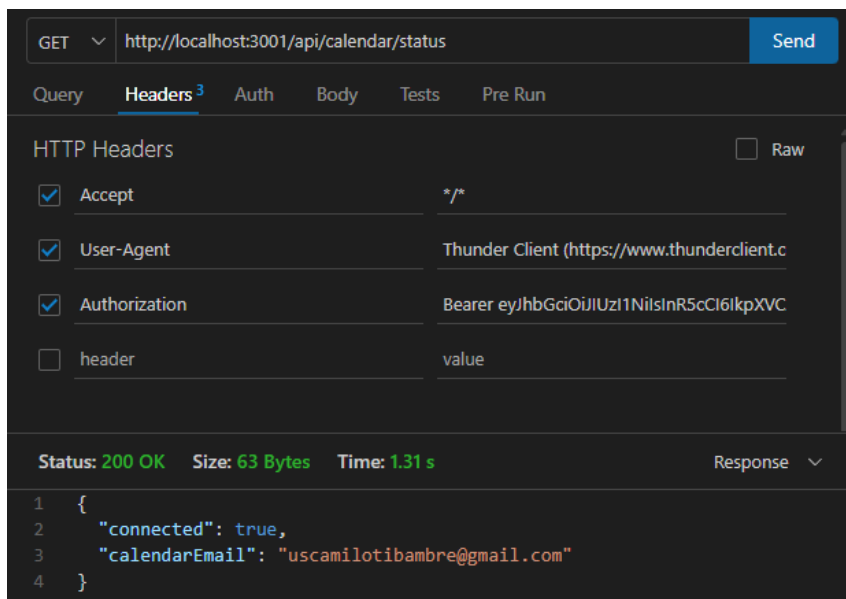
```

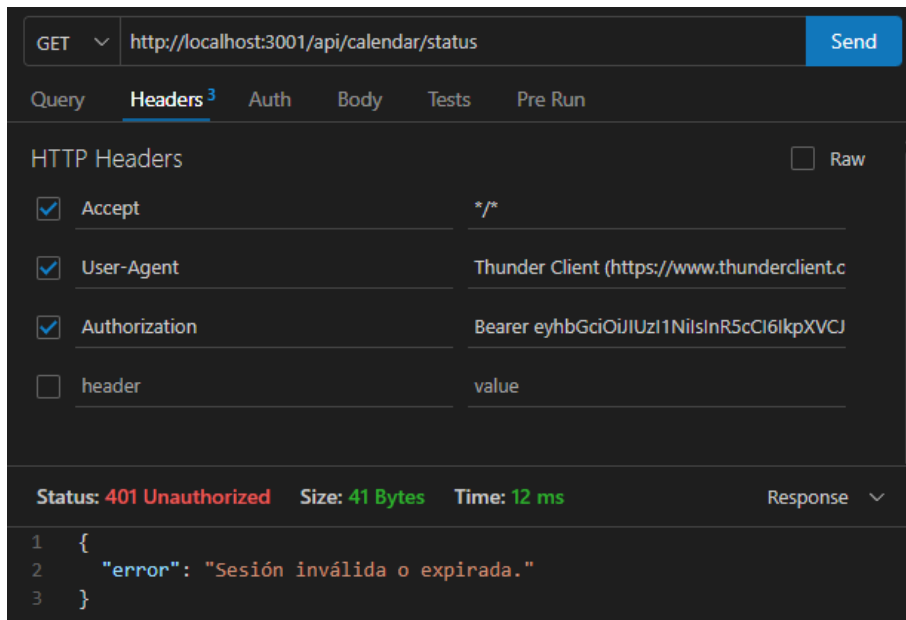
// Sin body. Lo consulta la pantalla de perfil al abrirse.

URL completa

`http://localhost:3001/api/calendar/status`

EVIDENCIA THUNDER CLIENT





GET ⌵ http://localhost:3001/api/calendar/status Send

Query Headers ³ Auth Body Tests Pre Run

HTTP Headers Raw

☒	Accept	*/*
☒	User-Agent	Thunder Client (https://www.thunderclient.c
☒	Authorization	Bearer eyhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ
☐	header	value

Status: **401 Unauthorized** Size: **41 Bytes** Time: **12 ms** Response ⌵

```

1 {
2   "error": "Sesión inválida o expirada."
3 }
```

2.3 GET /api/google/auth-url?action=login|link

GET

/api/google/auth-url?action=login|link

Vincula (o re-vincula) Google Calendar con un serverAuthCode obtenido del SDK nativo. Se usa desde "Vincular con Google" en el perfil, cuando el usuario ya tiene sesión.

REQUEST

```
GET /api/google/auth-url?action=login|link HTTP/1.1
Host: 192.168.0.6:3001
Authorization: Bearer {{JWT}}
Content-Type: application/json
```

URL completa

```
http://localhost:3001/api/google/auth-url?action=login|link
```

EVIDENCIA THUNDER CLIENT

GET <http://localhost:3001/api/google/auth-url?action=login|link> Send

Query Headers³ Auth Body Tests Pre Run

HTTP Headers ☐ Raw

☒	Accept	*/*
☒	User-Agent	Thunder Client (https://www.thunderclient.c
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC
☐	header	value

Status: 200 OK Size: 720 Bytes Time: 41 ms Response ▾

```

1 {
2   "url": "https://accounts.google.com/o/oauth2/v2/auth?client_id
   =3072212776-4tak504hvbj221drivoa905fd86tsh20.apps
   .googleusercontent.com&redirect_uri
   =http%3A%2F%2Flocalhost%3A3001%2Fapi%2Fgoogle%2Fcallback&response
   _type=code&scope=openid+email+profile+https%3A%2F%2Fwww
   .googleapis.com%2Fauth%2Fcalendar+https%3A%2F%2Fwww.googleapis
   .com%2Fauth%2Fcalendar.events&access_type
   =offline&include_granted_scopes=true&prompt=consent&state
   =eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
   .eyJhY3Rpb24iOiJsaW5rIiwiaWQiOiJQzLCJpYXQiOiJlE30TAyMTg0OTgsImV4cCI6
   MTc5MDIxOTA5OH0.6iumsERJHg8do6Sqdebs0PWsQfmZ74BwQM8Ci1wl13Y",

```

GET <http://localhost:3001/api/google/auth-url?action=login|link> Send

Query Headers³ Auth Body Tests Pre Run

HTTP Headers ☐ Raw

☒	Accept	*/*
☒	User-Agent	Thunder Client (https://www.thunderclient.c
☒	Authorization	Bearer eyhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ
☐	header	value

Status: 401 Unauthorized Size: 41 Bytes Time: 9 ms Response ▾

```

1 {
2   "error": "Sesión inválida o expirada."
3 }

```

2.4 DELETE /api/calendar/connect

DELETE

/api/calendar/connect

Desvincula Google Calendar del usuario autenticado y borra sus tokens guardados.

REQUEST

```

DELETE /api/calendar/connect HTTP/1.1
Host: 192.168.0.6:3001
Authorization: Bearer {{JWT}}

```

```
// Sin body. Botón "Desvincular Google Calendar" del perfil.
```

URL completa

```
http://localhost:3001/api/calendar/connect
```

EVIDENCIA THUNDER CLIENT

DELETE ⌵ http://localhost:3001/api/calendar/connect Send

Query **Headers**³ Auth Body Tests Pre Run

HTTP Headers ⌵ Raw

Header	Value
☒ Accept	*/*
☒ User-Agent	Thunder Client (https://www.thunderclient.c
☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC
☐ header	value

Status: 200 OK Size: 57 Bytes Time: 1.42 s Response ⌵

```

1 {
2   "mensaje": "Google Calendar desvinculado correctamente."
3 }
```

DELETE ⌵ http://localhost:3001/api/calendar/connect Send

Query **Headers**³ Auth Body Tests Pre Run

HTTP Headers ⌵ Raw

Header	Value
☒ Accept	*/*
☒ User-Agent	Thunder Client (https://www.thunderclient.c
☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ
☐ header	value

Status: 401 Unauthorized Size: 41 Bytes Time: 5 ms Response ⌵

```

1 {
2   "error": "Sesión inválida o expirada."
3 }
```

2.5 POST /api/calendar/sync

POST /api/calendar/sync

Sincroniza todas las órdenes visibles para el usuario como eventos en su Google Calendar, uno por cada Fecha_Limite (admin: todas · operario y cliente: las suyas).

REQUEST

```
POST /api/calendar/sync HTTP/1.1
Host: 192.168.0.6:3001
Authorization: Bearer {{JWT}}

// Sin body. Botón "Sincronizar Ahora" del perfil.
// Solo se crean eventos para órdenes que tengan Fecha_Limite.
```

URL completa

```
http://localhost:3001/api/calendar/sync
```

EVIDENCIA THUNDER CLIENT

POST <http://localhost:3001/api/calendar/sync> [Send](#)

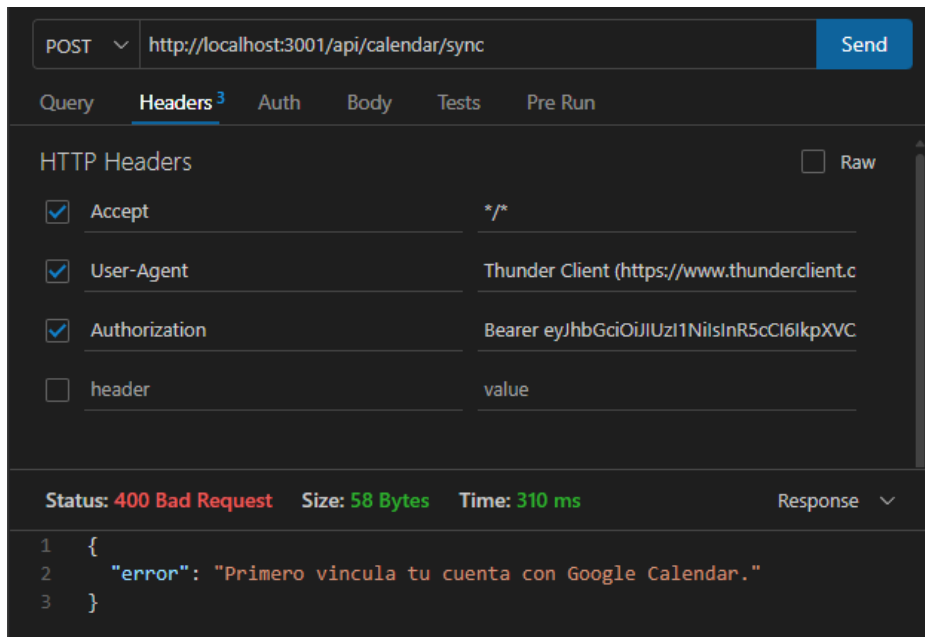
Query **Headers³** Auth Body Tests Pre Run

HTTP Headers ☐ Raw

☒	Accept	*/*
☒	User-Agent	Thunder Client (https://www.thunderclient.c
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC
☐	header	value

Status: **200 OK** Size: **42 Bytes** Time: **17.36 s** Response [▼](#)

```
1 {
2   "creados": 0,
3   "actualizados": 16,
4   "total": 16
5 }
```



2.6 POST /api/calendar/sync/:idOrden

POST

/api/calendar/sync/:idOrden

Sincroniza una sola orden con Google Calendar (por ejemplo, justo después de crearla o de cambiar su Fecha_Limite).

REQUEST

```

POST /api/calendar/sync/{{ID_ORDEN}} HTTP/1.1
Host: 192.168.0.6:3001
Authorization: Bearer {{JWT}}

// Sin body.
// :idOrden -> Id_Orden de la orden de producción (entero positivo).
```

URL completa

```
http://localhost:3001/api/calendar/sync/188
```

EVIDENCIA THUNDER CLIENT

POST ⌵ http://localhost:3001/api/calendar/sync/188 Send

Query **Headers³** Auth Body Tests Pre Run

HTTP Headers ⌵ ☐ Raw

☒	Accept	*/*
☒	User-Agent	Thunder Client (https://www.thunderclient.c
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC
☐	header	value

Status: 200 OK Size: 47 Bytes Time: 1.53 s Response ⌵

```
1 {  
2   "mensaje": "Orden sincronizada correctamente."  
3 }
```

POST ⌵ http://localhost:3001/api/calendar/sync/idOrden Send

Query **Headers³** Auth Body Tests Pre Run

HTTP Headers ⌵ ☐ Raw

☒	Accept	*/*
☒	User-Agent	Thunder Client (https://www.thunderclient.c
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC
☐	header	value

Status: 404 Not Found Size: 45 Bytes Time: 1.59 s Response ⌵

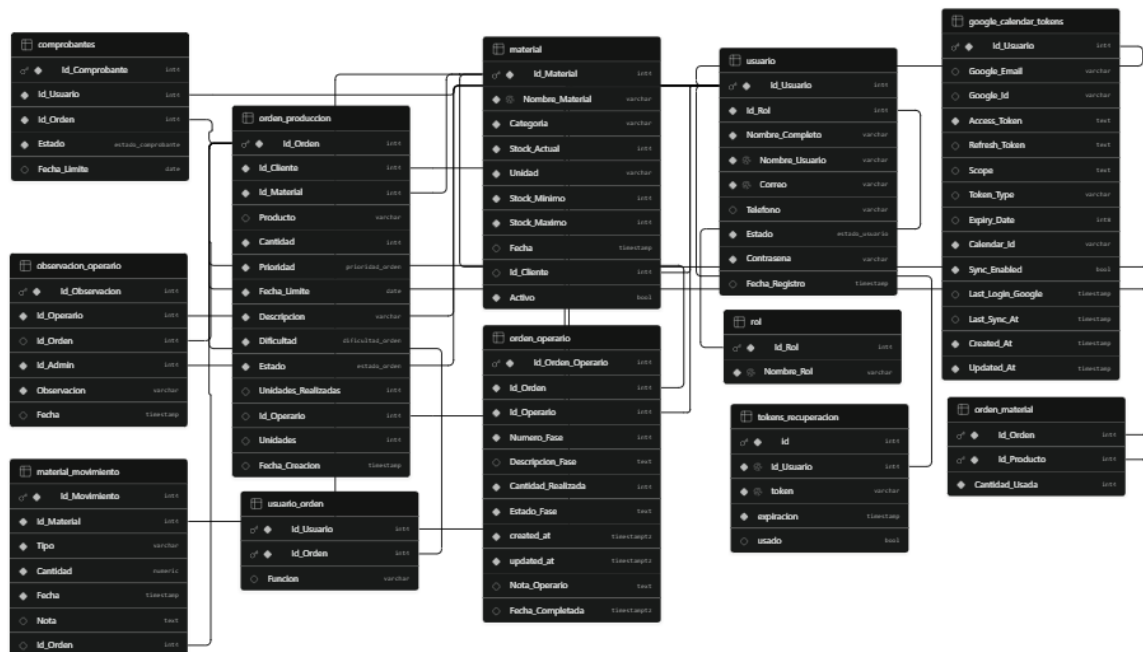
```
1 {  
2   "error": "Orden no encontrada o sin acceso."  
3 }
```

3. Diagrama Entidad-Relación

El módulo de Google opera sobre las tablas de usuarios y órdenes de Texticode. El login con Google reutiliza la tabla usuario (correo, rol y estado) y la vinculación de Calendar guarda, por cada Id_Usuario, los datos de la cuenta de Google.

Tabla	Campos
usuario	Id_Usuario (PK) · Nombre_Completo · Nombre_Usuario · Correo (UNIQUE) · Telefono · Estado · Id_Rol (FK)
rol	Id_Rol (PK) · Nombre_Rol (admin operario cliente)
orden_produccion	Id_Orden (PK) · Id_Operario (FK) · Id_Cliente (FK) · Id_Material (FK) · Producto · Cantidad · Prioridad · Estado · Unidades · Unidades_Realizadas · Fecha_Limite · Fecha_Creacion
Vinculación Google (por usuario)	Id_Usuario · refresh_token · calendarEmail · syncEnabled · calendarId
Evento de Calendar (por orden y usuario)	Id_Orden · Id_Usuario · eventId de Google (evita eventos duplicados al sincronizar)

DIAGRAMA



Relaciones clave

Origen	Relación	Destino	Descripción
rol	1 → N	usuario	Un rol agrupa múltiples usuarios; define qué órdenes ve cada uno al sincronizar
usuario	1 → N	orden_produccion	Un operario o cliente tiene múltiples órdenes asignadas

Origen	Relación	Destino	Descripción
usuario	1 → 1	Vinculación Google	Cada usuario tiene como máximo una cuenta de Google Calendar vinculada
orden_produccion	1 → N	Evento de Calendar	Una orden genera un evento por cada usuario que la sincroniza

4. Validaciones y Manejo de Errores

Todos los endpoints de Calendar comparten la validación del JWT de Texticode; el login con Google valida además el idToken contra Google. Los errores se devuelven como JSON con el campo error.

Escenario	Validación	HTTP
Falta idToken	El body de /api/auth/google/mobile no trae idToken	400 Bad Request
idToken inválido	Google no verifica el idToken contra el Web Client ID	401 Unauthorized
Sesión ausente o inválida	Header Authorization Bearer ausente, JWT inválido o expirado	401 Unauthorized
Cuenta inactiva	usuario.Estado distinto de activo	403 Forbidden
Usuario no registrado	No existe un usuario Texticode con el Correo de la cuenta de Google	404 Not Found
Falta serverAuthCode	El body de /api/calendar/connect no trae serverAuthCode	400 Bad Request
serverAuthCode rechazado	Código inválido, expirado o ya usado (un solo uso); redirect_uri_mismatch si se envía redirect_uri	400 Bad Request
Calendar no vinculado	No hay tokens guardados o la sincronización está desactivada	400 Bad Request
Error de Google Calendar	Algún request a la API de Google falla al crear o actualizar eventos	400 Bad Request
Orden no encontrada	:idOrden inexistente o no visible para el rol del usuario	404 Not Found
Error interno	catch → registro en el servidor	500 Server Error

GoogleAuthRepository.dart (app móvil)

```
late final GoogleSignIn _googleSignIn = GoogleSignIn(
  scopes: ['email', 'https://www.googleapis.com/auth/calendar.events'],
  serverClientId: serverClientId,
);
final account = await _googleSignIn.signIn();
final googleAuth = await account.authentication;
final serverAuthCode = account.serverAuthCode;
final res = await http.post(
  Uri.parse(ApiConstants.googleMobileLogin),
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({
    'idToken': googleAuth.idToken,
    'accessToken': googleAuth.accessToken,
    'serverAuthCode': serverAuthCode,
  }),
);
```